

ESMAD — Instituto Politécnico do Porto
TSIW — Linear Algebra and Analytic Geometry

Assignment Specifications

Graded Submission Specification for A01–A06

2025 / 2026

Python

JavaScript

NumPy / SciPy / Matplotlib math.js / Plotly.js / D3.js

Instructor: **Diogo Ribeiro**

dfr@esmad.ipp.pt

ORCID: [0009-0001-2022-7072](https://orcid.org/0009-0001-2022-7072)

github.com/diogoribeiro7/linear-algebra-with-python

Contents

1	General Instructions	4
1.1	Overview	4
1.2	Submission Procedure	4
1.3	Technical Requirements	4
1.3.1	Python Track	4
1.3.2	JavaScript Track	4
1.4	Repository Structure	5
1.5	Evaluation Schedule	5
1.6	Language Equivalence Table	5
1.7	Visualization Guidelines	6
1.8	Academic Integrity	6
1.9	Getting Help	7
2	Assignment Summary (A01–A06)	8
3	Assignment 01 — Matrices: Construction and Classification	9
3.1	Objective	9
3.2	Context	9
3.3	Learning Outcomes	9
3.4	Tasks	10
3.5	Deliverables	10
3.6	Evaluation Criteria	11
4	Assignment 02 — Matrix Operations and Property Verification	12
4.1	Objective	12
4.2	Context	12
4.3	Learning Outcomes	12
4.4	Tasks	13
4.5	Deliverables	13
4.6	Evaluation Criteria	14
5	Assignment 03 — Determinants and Invertibility	15
5.1	Objective	15
5.2	Context	15
5.3	Learning Outcomes	15
5.4	Tasks	16
5.5	Deliverables	16
5.6	Evaluation Criteria	17
6	Assignment 04 — Linear Systems: Gauss and Classification	18
6.1	Objective	18
6.2	Context	18
6.3	Learning Outcomes	18
6.4	Tasks	19
6.5	Deliverables	19
6.6	Evaluation Criteria	20

7	Assignment 05 — Cramer and LU Decomposition	21
7.1	Objective	21
7.2	Context	21
7.3	Learning Outcomes	21
7.4	Tasks	22
7.5	Deliverables	22
7.6	Evaluation Criteria	23
8	Assignment 06 — Analytic Geometry: Lines and Planes	24
8.1	Objective	24
8.2	Context	24
8.3	Learning Outcomes	24
8.4	Tasks	25
8.5	Deliverables	25
8.6	Evaluation Criteria	26

1 General Instructions

1.1 Overview

This document contains the graded submission specification for Assignments A01 through A06 of the Linear Algebra and Analytic Geometry course. These assignments are released every two weeks and must be submitted via Pull Request to the course GitHub repository before the stated deadline. Assignment A07 remains in the repository as separate teaching material and is intentionally outside the graded submission-control workflow.

Entrega em uma linguagem. Cada assignment pode ser realizado em Python **ou** JavaScript. Deve ser entregue apenas **uma** implementação (à escolha do estudante), e ela deve produzir os resultados pedidos.

1.2 Submission Procedure

1. You will be added as a **collaborator** to the course repository at <https://github.com/diogoribeiro7/linear-algebra-with-python>. Accept the invitation sent to your GitHub account.
2. Clone the repository to your local machine.
3. Inside `assignments/submissions/`, create a folder named with your student number (e.g., 40250001).
4. For each assignment, create a subfolder (01/, 02/, ...) containing **either** the Python **or** the JavaScript implementation.
5. Create a **branch** named with your student number and assignment (e.g., 40250001-A01).
6. Commit your changes, push, and open a **Pull Request** against the `develop` branch with a clear title (e.g., *Submission 40250001 — A01*).

1.3 Technical Requirements

1.3.1 Python Track

All Python code must run with **Python 3.11+** using the project dependencies managed by Poetry. Install the project environment from `packages/python` with `poetry install`. Use the utility modules in `packages/python/src/linalg_utils/` where appropriate — you are expected to import and use these functions rather than reimplementing them from scratch (unless the task explicitly asks for an implementation).

Key Python Libraries

`numpy` — array operations and linear algebra
`scipy` — LU decomposition, advanced routines
`matplotlib` — 2D and 3D visualization
`sympy` — symbolic computation (optional)

1.3.2 JavaScript Track

All JavaScript code must run with **Node.js 20+** or in a modern browser. The project includes a `package.json` in `packages/js`. Install the JavaScript dependencies from `packages/js` with `yarn install`.

Key JavaScript Libraries

mathjs — matrix operations, linear algebra, and symbolic math
plotly.js — interactive 2D and 3D visualization (browser)
d3.js — data-driven visualization (browser)
chart.js — charting (optional, for simpler plots)
numeric.js — numerical methods (optional alternative to mathjs)

1.4 Repository Structure

```

assignments/submissions/4025XXXX/
  01/
    README.md
    python/
      matrizes_tipos.py
    javascript/
      matrizes_tipos.js (or .mjs)
    reflexao.md
  02/
    README.md
    python/
      operacoes_matrizes.ipynb
    javascript/
      operacoes_matrizes.js
      operacoes_matrizes.html (for browser visualizations)
    reflexao.md
  ...
  
```

1.5 Evaluation Schedule

- **Test 1** (April 27): covers Assignments A1 through A5 (matrices, determinants, linear systems, Cramer/LU).
- **Test 2** (June 19): covers Assignment A6 (analytic geometry).

Each assignment includes a **reflexao.md** file where you write a short reflection connecting the computational work to the underlying mathematical concepts. This is graded for *depth of understanding*, not length. The reflection is shared between the chosen language option — you do not need to write two separate reflections.

Submission deadlines for A01 through A06 are configured in `course/config/deadlines.yml`. The dates shown in this document match that file: A01 — 2026-04-08, A02 — 2026-04-22, A03 — 2026-05-06, A04 — 2026-05-20, A05 — 2026-06-03, A06 — 2026-06-17. Assignment A07 is not listed there because it is teaching material only.

1.6 Language Equivalence Table

The following table maps the core operations for Python or JavaScript:

Operation	Python (NumPy)	JavaScript (math.js)
Create matrix	<code>np.array([[1,2],[3,4]])</code>	<code>math.matrix([[1,2],[3,4]])</code>
Zeros	<code>np.zeros((m,n))</code>	<code>math.zeros(m,n)</code>
Identity	<code>np.eye(n)</code>	<code>math.identity(n)</code>
Multiply	<code>A @ B</code>	<code>math.multiply(A, B)</code>
Transpose	<code>A.T</code>	<code>math.transpose(A)</code>
Determinant	<code>np.linalg.det(A)</code>	<code>math.det(A)</code>
Inverse	<code>np.linalg.inv(A)</code>	<code>math.inv(A)</code>
Solve $Ax = b$	<code>np.linalg.solve(A,b)</code>	<code>math.lusolve(A, b)</code>
Element access	<code>A[i,j]</code>	<code>A.get([i,j])</code> or <code>A._data[i][j]</code>
Shape	<code>A.shape</code>	<code>A.size()</code>
Allclose	<code>np.allclose(A, B)</code>	<code>math.deepEqual(math.round(A,10), math.round(B,10))</code>
Cross product	<code>np.cross(u, v)</code>	<code>math.cross(u, v)</code>
Dot product	<code>np.dot(u, v)</code>	<code>math.dot(u, v)</code>

1.7 Visualization Guidelines

Python Visualization

Use **matplotlib** for all plots. Save figures with `plt.savefig()`. For 3D plots, use `mpl_toolkits.mplot3d`.

JavaScript Visualization

Use **Plotly.js** for interactive 2D/3D plots in the browser. Create an `.html` file that loads Plotly from CDN and renders the plots. For static output, you may also use **canvas**-based rendering with Node.js. Console-based outputs (tables, logs) should be produced with `console.table()` or formatted `console.log()`.

1.8 Academic Integrity

You may discuss ideas with classmates, but all code and written reflections must be your own. Submissions that contain copied code or text will receive a zero grade. Using AI tools to generate solutions is not permitted — you must demonstrate your own understanding.

1.9 Getting Help

If you encounter issues running the code or understanding the assignment tasks, file a GitHub Issue in the repository or contact the instructor at dfr@esmad.ipp.pt. Include the error message, the commands you ran, and the relevant file paths.

2 Assignment Summary (A01–A06)

#	Title	Module	Difficulty	Deadline
A1	Matrices: Construction and Classification	Matrices	Introductory	2026-04-08
A2	Matrix Operations and Property Verification	Operations, Transpose	Introductory	2026-04-22
A3	Determinants and Invertibility	Determinants, Inverse	Intermediate	2026-05-06
A4	Linear Systems: Gauss and Classification	Gaussian Elimination	Intermediate	2026-05-20
A5	Cramer and LU Decomposition	Cramer, LU	Intermediate	2026-06-03
A6	Analytic Geometry: Lines and Planes	Analytic Geometry	Intermediate	2026-06-17

Assignments A01 through A06 require implementation in **either Python or JavaScript**. The evaluation is based on the chosen language unless otherwise noted. Assignment A07 remains available in the repository as separate teaching material and is outside the PR-based submission workflow.

3 Assignment 01 — Matrices: Construction and Classification

Module	Matrices
Difficulty	Introductory
Deadline	2026-04-08
Submission	assignments/submissions/4025XXX/01/

3.1 Objective

Understand the structure of special matrix types, access individual elements programmatically, and build reusable classification functions using NumPy (Python) and math.js (JavaScript).

3.2 Context

Matrices are the foundational data structure in linear algebra. Before performing operations or solving systems, you must be comfortable creating, inspecting, and classifying matrices. This assignment ensures you have a solid practical foundation in the chosen language.

3.3 Learning Outcomes

- Create special matrices (zero, identity, diagonal, triangular, symmetric).
- Access elements by index and inspect dimensions.
- Write reusable functions for matrix classification in Python or JavaScript.
- Handle dimension mismatch errors gracefully in the chosen language.

3.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] Create the following matrices: a 3×4 zero matrix, a 4×4 identity matrix, a 3×3 diagonal matrix with entries $[2, 5, -1]$, an upper triangular 3×3 matrix, a lower triangular 3×3 matrix, and a symmetric matrix built from $M + M^T$ for a matrix M of your choice.	Printed matrices with labels
T2	[Python or JS] For each matrix created in T1, print its dimensions (rows $\times$ columns), the element at position $a_{2,3}$ (row 2, column 3, using 0-based indexing), and verify the result makes sense given the matrix structure.	Element values and dimensions
T3	[Python or JS] Write a function <code>classificar_matriz(A)</code> that receives a matrix and returns a list of labels from: <code>square</code> , <code>rectangular</code> , <code>zero</code> , <code>identity</code> , <code>diagonal</code> , <code>symmetric</code> , <code>upper_triangular</code> , <code>lower_triangular</code> . Test it on all matrices from T1.	Function code and test results
T4	[Python or JS] Demonstrate what happens when you try to add two matrices with incompatible dimensions (e.g., $2 \times 3 + 4 \times 2$) and when you try to multiply matrices with incompatible inner dimensions. Catch and print the errors.	Error messages with explanation
T5	Write a short reflection (<code>reflexao.md</code> , 150–250 words) explaining why the identity matrix simultaneously satisfies multiple classifications (diagonal, symmetric, triangular). Use set-theoretic language. Additionally, note any differences you observed between the Python or JavaScript APIs for matrix handling.	<code>reflexao.md</code> file

3.5 Deliverables

```
assignments/submissions/4025XXX/01/
  README.md
  python/
    matrizes_tipos.py
  javascript/
    matrizes_tipos.js
  reflexao.md
```

Hints

[Python] Use `np.zeros`, `np.eye`, `np.diag`, `np.triu`, `np.tril` for matrix creation. For classification, `np.allclose` is your best friend for floating-point comparisons. Test edge cases like 1×1 matrices.

[JavaScript] Use `math.zeros(m,n)`, `math.identity(n)`, `math.diag([...])` for matrix creation. For extracting triangular parts, you will need to write helper functions since `math.js` does not have built-in `triu/tril`. For comparisons, use `math.deepEqual` combined with `math.round` for floating-point tolerance. Error handling uses `try { ... } catch (e) { ... }`.

3.6 Evaluation Criteria

Correctness of matrix creation and element access (the chosen language)	30%
Quality and completeness of the classification function (the chosen language)	30%
Error handling demonstration (the chosen language)	15%
Reflection depth and clarity	25%

4 Assignment 02 — Matrix Operations and Property Verification

Module	Operations, Transpose
Difficulty	Introductory
Deadline	2026-04-22
Submission	<code>assignments/submissions/4025XXX/02/</code>

4.1 Objective

Practice matrix arithmetic operations and computationally verify fundamental algebraic properties such as non-commutativity, transpose identities, and the linear combination interpretation of matrix-vector products — in Python or JavaScript.

4.2 Context

Matrix algebra differs from scalar algebra in important ways. Multiplication is generally not commutative, and the transpose operation interacts with products in a specific order-reversing fashion. Verifying these properties computationally in two different languages builds deep intuition and cross-platform fluency.

4.3 Learning Outcomes

- Perform matrix addition, subtraction, and scalar multiplication.
- Compute matrix products respecting dimension rules.
- Verify non-commutativity and compute commutators.
- Prove transpose properties computationally.
- Interpret Ax as a linear combination of the columns of A .

4.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] Define two 2×3 matrices A and B . Compute and print $A + B$, $A - B$, and $3A$. Verify that the sum preserves dimensions using an assertion.	Computed results with verification
T2	[Python or JS] Define a 3×2 matrix C . Compute AC (2×3 times 3×2) and CA (3×2 times 2×3). Print both results and explain why they have different dimensions and are not equal.	Products with dimensional analysis
T3	[Python or JS] For two 2×2 matrices P and Q , compute PQ , QP , and the commutator $[P, Q] = PQ - QP$. Show that $PQ \neq QP$ in general, but that $PI = IP = P$ for the identity matrix.	Commutator and identity verification
T4	[Python or JS] Verify the three transpose properties computationally: (a) $(A^\top)^\top = A$; (b) $(A + B)^\top = A^\top + B^\top$; (c) $(AC)^\top = C^\top A^\top$. Use approximate comparison for each.	Boolean verification of each property
T5	[Python or JS] Generate a random 3×3 matrix M and construct $S = M + M^\top$. Verify that S is symmetric. Then define a 3×3 system matrix and a vector $\mathbf{x}$, compute $A\mathbf{x}$, and show that $A\mathbf{x}$ equals the linear combination $x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + x_3\mathbf{a}_3$ where $\mathbf{a}_i$ are the columns of A .	Symmetry proof and linear combination
T6	Write a <code>reflexao.md</code> (150–250 words) explaining the geometric or algebraic significance of non-commutativity in matrix multiplication. Compare how the two languages express these operations.	<code>reflexao.md</code> file

4.5 Deliverables

```

assignments/submissions/4025XXX/02/
  README.md
  python/
    operacoes_matrizes.ipynb
  javascript/
    operacoes_matrizes.js
    operacoes_matrizes.html    (for any browser visualizations)
  reflexao.md

```

Hints

[Python] Remember that `@` is the matrix multiplication operator in NumPy. Use `np.allclose` instead of `==` for floating-point comparisons. The linear combination view of $A\mathbf{x}$ is fundamental to understanding column spaces.

[JavaScript] Use `math.multiply(A, B)` for matrix multiplication, `math.add`, `math.subtract` for arithmetic. For random matrices, use `math.random([3,3])`. Access columns with `math.column(A, j)`. Use `math.deepEqual(math.round(X, 10), math.round(Y, 10))` instead of strict equality.

4.6 Evaluation Criteria

Correct operations and dimensional awareness	25%
Property verification completeness	25%
Commutator and non-commutativity analysis	20%
Linear combination demonstration	15%
Reflection quality	15%

5 Assignment 03 — Determinants and Invertibility

Module	Determinants, Inverse
Difficulty	Intermediate
Deadline	2026-05-06
Submission	<code>assignments/submissions/4025XXX/03/</code>

5.1 Objective

Implement determinant computation using multiple methods (direct formula, Sarrus, Gaussian elimination, cofactor expansion), explore invertibility conditions, and compute matrix inverses programmatically in Python or JavaScript.

5.2 Context

The determinant is a scalar that encodes whether a matrix is invertible and how it scales volumes. Understanding multiple computation methods helps you appreciate both the theory and the practical trade-offs between algorithms.

5.3 Learning Outcomes

- Compute determinants via the 2×2 formula, Sarrus' rule, Gaussian elimination, and cofactor expansion.
- Verify determinant properties (product rule, transpose, scalar multiple, row operations).
- Test invertibility and compute condition numbers.
- Compute inverses via direct formula and Gauss-Jordan elimination.

5.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] Implement <code>det_2x2(A)</code> for 2×2 matrices and <code>det_3x3_sarrus(A)</code> for 3×3 matrices. Test each on at least two matrices (one invertible, one singular) and compare with the built-in determinant.	Functions with test output
T2	[Python or JS] Implement a Gaussian elimination-based determinant function <code>det_gauss(A)</code> that returns both the determinant value and intermediate row echelon steps. Test on a 4×4 matrix.	Function with step-by-step output
T3	[Python or JS] Verify the following properties computationally using random matrices: $\det(AB) = \det(A)\det(B)$; $\det(A^T) = \det(A)$; $\det(kA) = k^n \det(A)$; row swap negates the determinant; adding a multiple of one row to another preserves the determinant.	Verification of 5 properties
T4	[Python or JS] Write a function <code>eh_invertivel(A)</code> that checks invertibility. Apply it to three matrices: one clearly invertible, one singular, and one ill-conditioned (with condition number $> 10^8$). Report the condition number for each.	Invertibility report with cond. numbers
T5	[Python or JS] Implement <code>inversa_gauss_jordan(A)</code> that computes the inverse via augmented matrix $[A I]$ reduction. Test on a 3×3 and a 4×4 matrix, verifying that $A \cdot A^{-1} = I$.	Inverse computation with verification
T6	[Python or JS] Create a visualization showing how a 2×2 matrix transforms the unit square into a parallelogram. Display the area scale factor (which equals $ \det(A) $) in the plot title. Save the figure.	Saved figure <code>determinante_area.png/html</code>
T7	Write <code>reflexao.md</code> (200–300 words) explaining the geometric meaning of the determinant and why $\det(A) = 0$ implies non-invertibility.	<code>reflexao.md</code> file

5.5 Deliverables

```

assignments/submissions/4025XXX/03/
  README.md
  python/
    determinantes.ipynb
    determinante_area.png
  javascript/
    determinantes.js
    determinante_area.html    (interactive Plotly visualization)
  reflexao.md

```

Hints

[Python] For Gauss-Jordan, augment $[A|I]$ and reduce to $[I|A^{-1}]$. Use partial pivoting for numerical stability. The condition number is `np.linalg.cond(A)`. For visualization, transform the vertices of the unit square $[0,0]$, $[1,0]$, $[1,1]$, $[0,1]$.

[JavaScript] Use `math.det(A)` for verification. For the Gauss-Jordan implementation, work with 2D arrays (not `math.js` matrices) for easier row manipulation. Compute condition number manually as $\|A\| \cdot \|A^{-1}\|$ using `math.norm`. For visualization, use Plotly.js shapes or SVG to draw the unit square and its transformation.

5.6 Evaluation Criteria

Determinant implementations and correctness	25%
Property verification	15%
Invertibility analysis with condition numbers	15%
Gauss-Jordan inverse	20%
Visualization quality	10%
Reflection depth	15%

6 Assignment 04 — Linear Systems: Gauss and Classification

Module	Gaussian Elimination
Difficulty	Intermediate
Deadline	2026-05-20
Submission	<code>assignments/submissions/4025XXX/04/</code>

6.1 Objective

Solve and classify linear systems using Gaussian elimination with partial pivoting, implement back substitution, handle all three system types (SPD, SPI, SI), and visualize solution geometry — in Python or JavaScript.

6.2 Context

Solving $Ax = b$ is the central problem of linear algebra. Gaussian elimination reduces the system to echelon form, from which you can read off the solution (if unique), parametric families (if infinite), or contradictions (if impossible). The Rouché–Capelli theorem provides the theoretical framework.

6.3 Learning Outcomes

- Implement Gaussian elimination with partial pivoting and step tracking.
- Classify systems as SPD (unique), SPI (infinite), or SI (impossible).
- Extract parametric solutions for underdetermined systems.
- Visualize 2D systems as intersecting lines and 3D systems as intersecting planes.

6.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] Implement <code>escalonar(Ab)</code> that takes an augmented matrix and returns the echelon form with a log of row operations. Implement <code>substituicao_retroativa(U, b)</code> for back substitution. Solve: $2x + y - z = 8$, $-3x - y + 2z = -11$, $-2x + y + 2z = -3$.	Step-by-step elimination output
T2	[Python or JS] Construct an indeterminate system (SPI) where the second row is a multiple of the first. Show that echelon form reveals a free variable. Produce at least three particular solutions by assigning different values to the free parameter.	Echelon form and parametric solutions
T3	[Python or JS] Construct an impossible system (SI). Show that echelon form reveals a contradiction ($0 = \text{nonzero}$). Explain geometrically why the system has no solution.	Echelon form with contradiction
T4	[Python or JS] Write <code>classificar_sistema(A, b)</code> that uses rank analysis (Rouché–Capelli) to return 'SPD', 'SPI', or 'SI'. Test on at least 5 systems covering all three types, including 2×2 , 3×3 , and 4×4 examples.	Classification table for 5+ systems
T5	[Python or JS] For an SPI system with 3 unknowns and 2 equations, write out the general parametric solution explicitly. Verify that substituting specific parameter values satisfies the original equations.	Parametric solution with verification
T6	[Python or JS] Compare your Gauss implementation with the built-in solver for SPD systems. Show that they produce the same result. Then demonstrate what happens when the built-in solver receives a singular system.	Comparison with built-in solver
T7	[Python or JS] Create two visualizations: (a) three 2D systems shown as pairs of lines illustrating unique, infinite, and no solution; (b) a 3D visualization showing three planes intersecting at a point.	Saved figures
T8	Write <code>reflexao.md</code> (200–300 words) explaining the Rouché–Capelli theorem and its connection to Gaussian elimination.	<code>reflexao.md</code> file

6.5 Deliverables

```

assignments/submissions/4025XXX/04/
  README.md
  python/
    sistemas_lineares.ipynb
  *.png
  javascript/
    sistemas_lineares.js
    sistemas_lineares.html    (interactive visualizations)
  reflexao.md

```

Hints

[Python] Use `np.linalg.matrix_rank` for classification. For parametric solutions, identify pivot vs. free variables from the echelon form. In 3D plots, use `plot_surface` with `alpha` for transparency.

[JavaScript] Compute matrix rank manually by counting non-zero rows in the echelon form (math.js does not have a built-in rank function). For 3D visualizations, use Plotly.js with `type: 'surface'` or `type: 'mesh3d'` traces. The built-in solver is `math.lusolve(A, b)`.

6.6 Evaluation Criteria

Gaussian elimination and back substitution	25%
System classification correctness	20%
Parametric solutions	15%
Built-in solver comparison	10%
Visualizations	15%
Reflection	15%

7 Assignment 05 — Cramer and LU Decomposition

Module	Cramer, LU
Difficulty	Intermediate
Deadline	2026-06-03
Submission	<code>assignments/submissions/4025XXX/05/</code>

7.1 Objective

Compare Cramer's rule with Gaussian elimination and LU decomposition, analyze computational costs empirically, and demonstrate the efficiency of LU for multiple right-hand sides — in Python or JavaScript.

7.2 Context

Different solution methods have different strengths. Cramer's rule provides elegant closed-form solutions but scales terribly. LU decomposition pays a one-time factorization cost and then solves additional systems cheaply. Understanding these trade-offs is essential for applied work.

7.3 Learning Outcomes

- Implement Cramer's rule for 2×2 and 3×3 systems.
- Use LU decomposition (SciPy in Python, manual or library-based in JavaScript).
- Benchmark methods across matrix sizes.
- Solve multiple right-hand sides efficiently with LU.
- Visualize matrix factors as heatmaps.

7.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] Implement <code>cramer_2x2(A, b)</code> and <code>cramer_3x3(A, b)</code> . Solve at least one system with each and compare results with the built-in solver.	Cramer solutions with verification
T2	[Python or JS] Compute the LU decomposition of a 3×3 matrix. Print P , L , and U . Verify that $PA = LU$. Compute $\det(A)$ from the diagonal of U .	LU factors with verification
T3	[Python or JS] Benchmark Cramer, your Gauss implementation, and the built-in solver for matrix sizes $n = 2, 3, 5, 10, 20, 50, 100$. Use timing mechanisms appropriate for each language. Present results in a table and bar chart.	Benchmark table and figure
T4	[Python or JS] Given a fixed matrix A and three different right-hand sides b_1, b_2, b_3 , solve all three using a single LU factorization. Compare the total time with solving each from scratch.	Multi-RHS solution with timing
T5	[Python or JS] Create a heatmap visualization showing the structure of A , L , and U side by side, with numerical values annotated in each cell.	Saved figure <code>lu_heatmap.png/html</code>
T6	Write <code>reflexao.md</code> (200–300 words) comparing the three methods (Cramer, Gauss, LU) in terms of computational complexity, practical use cases, and numerical stability.	<code>reflexao.md</code> file

7.5 Deliverables

```

assignments/submissions/4025XXX/05/
  README.md
  python/
    cramer_lu.ipynb
    lu_heatmap.png
  javascript/
    cramer_lu.js
    cramer_lu.html          (interactive benchmarks and heatmap)
  reflexao.md

```

Hints

[Python] For Cramer, replace column i of A with b and compute the determinant. Use `scipy.linalg.lu_factor` and `lu_solve` for efficient LU operations. For benchmarking, use `min(timeit.repeat(...))`.

[JavaScript] For LU decomposition, implement your own or use the `numeric.js` library (`numeric.LU(A)`). For Cramer, use `math.det` combined with column replacement. For benchmarking, use `performance.now()` (browser) or `process.hrtime.bigint()` (Node.js). For heatmaps, use Plotly.js with `type: 'heatmap'` and `text` annotations.

7.6 Evaluation Criteria

Cramer implementation	15%
LU decomposition and verification	20%
Benchmarking quality and presentation	25%
Multiple RHS demonstration	15%
Visualization	10%
Reflection	15%

8 Assignment 06 — Analytic Geometry: Lines and Planes

Module	Analytic Geometry
Difficulty	Intermediate
Deadline	2026-06-17
Submission	<code>assignments/submissions/4025XXX/06/</code>

8.1 Objective

Represent lines and planes in parametric and Cartesian forms, classify relative positions (intersecting, parallel, coincident, skew), and compute intersections using linear algebra tools — in Python or JavaScript.

8.2 Context

Analytic geometry connects geometric objects (lines, planes) with algebraic representations. Classifying relative positions of lines and planes is equivalent to analyzing the solutions of associated linear systems, directly applying the theory from previous assignments.

8.3 Learning Outcomes

- Generate parametric representations of lines in $\mathbb{R}^2$ and $\mathbb{R}^3$.
- Convert between Cartesian and parametric plane representations.
- Classify relative positions using cross products and rank analysis.
- Compute line-plane intersections.
- Visualize geometric configurations in 2D and 3D.

8.4 Tasks

#	Task Description	Expected Output
T1	[Python or JS] For three pairs of 2D lines (one intersecting, one parallel, one coincident), implement a function <code>posicao_relativa_retas_2d</code> that classifies them. Visualize all three cases in a single figure with three subplots.	Classification function and figure
T2	[Python or JS] Extend to 3D with four cases: intersecting, parallel, coincident, and skew lines. Implement <code>posicao_relativa_retas_3d</code> and visualize in a 2×2 subplot grid.	3D line classification and figure
T3	[Python or JS] Implement <code>plano_cartesiano_para_parametrico(a, b, c, d)</code> that converts $ax + by + cz = d$ into parametric form (point + two direction vectors). Test on two different planes.	Conversion function with output
T4	[Python or JS] Classify pairs of planes as secant, parallel, or coincident. Implement <code>posicao_relativa_planos</code> and test on three pairs.	Plane classification results
T5	[Python or JS] Implement <code>intersecao_reta_plano</code> that computes the intersection point between a line and a plane. Test with three cases: intersecting, parallel, and contained.	Intersection computation
T6	[Python or JS] Demonstrate dot product tests for parallelism and perpendicularity: two perpendicular lines, a line parallel to a plane, two perpendicular planes.	Dot product verification
T7	[Python or JS] Create a comprehensive 3D visualization showing a plane, a line intersecting it, a parallel line, the plane normal vector, and the intersection point marked.	Saved figure <code>geometria_resumo.png/html</code>
T8	Write <code>reflexao.md</code> (200–300 words) explaining the role of direction and normal vectors in determining relative positions.	<code>reflexao.md</code> file

8.5 Deliverables

```

assignments/submissions/4025XXX/06/
  README.md
  python/
    geometria_analitica.ipynb
    geometria_resumo.png
  javascript/
    geometria_analitica.js
    geometria_analitica.html    (interactive 3D with Plotly)
  reflexao.md

```

Hints

[Python] For 2D line classification, the cross product of direction vectors (as a scalar in 2D) tells you if lines are parallel. For line-plane intersection, substitute the parametric line equation into the plane equation and solve for t . Use `np.cross` for 3D cross products.

[JavaScript] Use `math.cross(u, v)` for cross products and `math.dot(u, v)` for dot prod-

ucts. For 3D visualizations, Plotly.js excels with `type: 'scatter3d'` (lines and points) and `type: 'mesh3d'` or `type: 'surface'` (planes). Define planes as meshgrids computed with nested loops or array comprehensions.

8.6 Evaluation Criteria

2D line classification and visualization	15%
3D line classification	15%
Plane conversions and classification	15%
Line-plane intersection	15%
Dot product demonstrations	10%
Comprehensive visualization	15%
Reflection	15%